WEEK 2
Provider-Agnostic LLM Client

1. Project Overview

The objective of Week 2 is to build a provider-agnostic Large Language Model (LLM) client. The application sends a prompt to a cloud-based Gemini model through the Google Gen AI SDK, and the code also includes a provider for a locally running Llama 3.2 model through Ollama. The rest of the application talks to one common client interface, so the provider can be changed without rewriting the application logic.

2. Objectives

- Understand abstraction and inheritance using Python object-oriented programming.
- Define a common LLM provider interface with an abstract base class.
- Connect to Google Gemini using the Google Gen AI Python SDK.
- Handle temporary Gemini errors (503 / UNAVAILABLE) with a simple retry.
- Connect to a local Ollama model through its HTTP API.
- Load API credentials securely from a .env file.
- Run the interactive chat application and verify the Gemini connection with a test script.

3. Tools and technologies

Tool / Library	Purpose
Python 3	Programming language (3.10 or newer; project environment uses 3.14)
PyCharm	Code editor and integrated terminal
uv	Dependency and virtual-environment manager (uv.lock, pyproject.toml)
Google Gemini API	Cloud-based LLM provider
google-genai	Python SDK for Gemini
Ollama	Runs the Llama 3.2 model locally (used by OllamaProvider)
requests	Sends HTTP requests to Ollama
python-dotenv	Loads environment variables from .env
pytest	Testing tool listed in requirements

4. Project Folder Structure

The project folder is named week-2. It contains the following files and folders:

```
week-2/
├── .venv/              # Python virtual environment
├── .env                # Gemini API key (kept private)
├── .gitignore          # Keeps .env and .venv out of Git
├── llm_client.py       # Provider classes and common client
├── main.py             # Program entry point (chat loop)
├── pyproject.toml      # Project metadata and dependencies
├── requirements.txt    # Python dependencies
├── test_client.py      # Gemini connection test
├── uv.lock             # Locked dependency versions
└── README.txt          # Short setup notes
```

The __pycache__ and .pytest_cache folders may also appear automatically after Python or pytest runs. They store compiled and cached files and do not need to be edited. The .idea folder is created by PyCharm.

Figure 1. week-2 project folder and files in PyCharm.

5. Environment Setup on Windows

1. Open the week-2 folder in PyCharm.
2. Open the integrated terminal (View > Tool Windows > Terminal).
3. Install dependencies with uv: `uv sync`
4. Or create a virtual environment manually: `python -m venv .venv`
5. Activate it: `.venv\Scripts\activate`. The terminal should show (.venv).
6. Install dependencies: `pip install -r requirements.txt`
7. Optional (only for the Ollama provider): install Ollama, then download the model with `ollama pull llama3.2`. The Ollama service listens on http://localhost:11434.

6. Configuration: .env

The Gemini API key is stored in a .env file so that it is not hard-coded into the Python source. Replace the placeholder below with the API key created for your Google AI Studio project. The .env file is listed in .gitignore and must never be uploaded to GitHub or shown in screenshots.

```
GEMINI_API_KEY=YOUR_GEMINI_API_KEY
```

7. Dependencies: requirements.txt

Install all listed packages from the project terminal with:

```
python-dotenv
pytest
google-genai
requests

pip install -r requirements.txt
```

8. Source Code

8.1 llm_client.py

This file defines the abstract provider interface, the Gemini implementation (with a retry for temporary 503 / UNAVAILABLE errors), the Ollama implementation, and the common LLMClient. Both providers implement the same generate() method.

```
from abc import ABC, abstractmethod
from google import genai
import requests
import time


# Common interface
class LLMProvider(ABC):

    @abstractmethod
    def generate(self, prompt):
        pass


# Gemini cloud provider
class GeminiProvider(LLMProvider):

    def __init__(self, api_key):
        self.client = genai.Client(api_key=api_key)
        self.model = "gemini-3.5-flash-lite"

    def generate(self, prompt):

        for attempt in range(3):
            try:
                response = self.client.models.generate_content(
                    model=self.model,
                    contents=prompt
                )

                if not response.text:
                    raise RuntimeError("Gemini returned an empty response.")

                return response.text

            except Exception as e:
                error_message = str(e)

                if ("503" in error_message or "UNAVAILABLE" in error_message) and attempt < 2:
                    print("Gemini is busy. Retrying...")
                    time.sleep(3)
                else:
```

```
                    raise


# Ollama local provider
class OllamaProvider(LLMProvider):

    def __init__(self, model="llama3.2"):
        self.model = model
        self.url = "http://localhost:11434/api/generate"

    def generate(self, prompt):

        response = requests.post(
            self.url,
            json={
                "model": self.model,
                "prompt": prompt,
                "stream": False
            },
            timeout=120
        )

        response.raise_for_status()

        return response.json()["response"]


# Common client
class LLMClient:

    def __init__(self, provider):
        self.provider = provider

    def generate(self, prompt):
        return self.provider.generate(prompt)
```

8.2 main.py

The main program loads the .env file, checks that the API key exists, creates the Gemini provider, and runs an interactive chat loop. Typing exit ends the program, and empty input is rejected with a message.

```
import os
from dotenv import load_dotenv
from llm_client import GeminiProvider

load_dotenv()

api_key = os.getenv("GEMINI_API_KEY")

if not api_key:
    raise ValueError("GEMINI_API_KEY not found in .env file")


def main():
    print("===== WEEK 2: GENERATIVE AI APPLICATION =====")

    try:
        llm = GeminiProvider(api_key)

        print("\nGemini AI is ready!")
        print("Type 'exit' to quit.")

        while True:
            question = input("\nYou: ").strip()
```

```
            if question.lower() == "exit":
                print("Thank you! Goodbye.")
                break

            if not question:
                print("Please enter a question.")
                continue

            try:
                print("\nGemini is thinking...")
                response = llm.generate(question)
                print("\nGemini:", response)

            except Exception as e:
                print("\nError:", e)

    except Exception as e:
        print("Initialization Error:", e)


if __name__ == "__main__":
    main()
```

8.3 test_client.py

This test loads the API key, builds a GeminiProvider wrapped in the common LLMClient, sends a short test message, and prints the response. If anything fails, it prints TEST FAILED with the reason.

```
import os

from dotenv import load_dotenv

from llm_client import GeminiProvider, LLMClient

load_dotenv()


def test_gemini_connection():
    api_key = os.getenv("GEMINI_API_KEY")
    if not api_key:
        raise ValueError(
            "GEMINI_API_KEY was not found. Check your .env file."
        )

    print("Connecting to Gemini...")
    provider = GeminiProvider(api_key)
    client = LLMClient(provider)

    print("Sending test message...")
    answer = client.generate("Reply with: Gemini connection successful.")

    print("\nGemini response:")
    print(answer)


if __name__ == "__main__":
    try:
        test_gemini_connection()
    except Exception as exc:
        print(f"\nTEST FAILED: {exc}")
```

9. Running the Application

Run these commands in the PyCharm terminal from inside the week-2 folder:

```
.venv\Scripts\activate
python main.py
```

Or, with uv:

```
uv run python main.py
```

The program prints a welcome banner and the message “Gemini AI is ready!”. Then:

- type a question at the You: prompt and the answer is displayed after Gemini:
- type exit to quit the program.

To verify the connection before chatting, run the test script:

```
python test_client.py
```

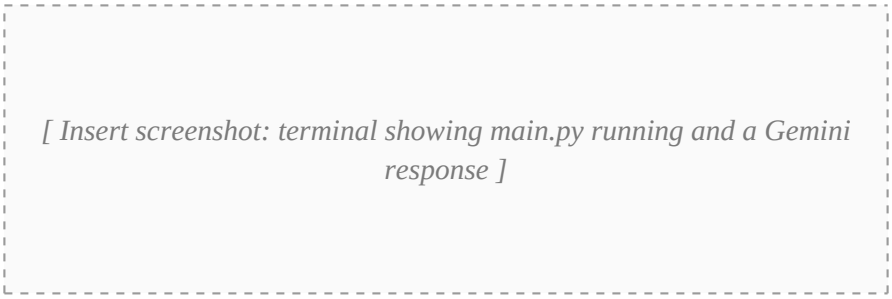

Figure 2. Running main.py with the Gemini provider.

10. Result

The provider-agnostic LLM client was implemented using Python OOP. The application loads the Gemini API key from the .env file, sends each question through the common LLMClient interface to the Gemini provider, and prints the generated answer. The code also includes an Ollama provider for the local Llama 3.2 model, environment-based API-key configuration, automatic retry for temporary Gemini outages, and a connection test for the common LLMClient interface.

11. Conclusion

This project demonstrates how a single application can support multiple LLM providers through a common abstraction. By separating provider-specific code from the client logic, the design is easier to maintain and extend. Another provider can be added by implementing the LLMProvider interface without changing how the main application calls generate().